

# **Отчёт по лабораторной работе №7**

**Математические основы защиты информации и информационной безопасности**

Ли Хан

# Содержание

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                                | <b>5</b>  |
| <b>2</b> | <b>Модуль реализации алгоритма</b>                | <b>6</b>  |
| 2.1      | Модуль реализации алгоритма . . . . .             | 6         |
| 2.2      | Шаги реализации: . . . . .                        | 6         |
| 2.2.1    | Модуль реализации алгоритма компилятора . . . . . | 8         |
| 2.2.2    | Модуль реализации алгоритма компилятора . . . . . | 9         |
| 2.2.3    | Модуль реализации алгоритма компилятора . . . . . | 10        |
| 2.2.4    | результат . . . . .                               | 11        |
| <b>3</b> | <b>Вывод</b>                                      | <b>12</b> |

## Список иллюстраций

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 2.1 | Модуль реализации алгоритма компилятора . . . . . | 8  |
| 2.2 | Модуль реализации алгоритма компилятора . . . . . | 9  |
| 2.3 | Модуль реализации алгоритма компилятора . . . . . | 10 |
| 2.4 | Модуль реализации алгоритма результат . . . . .   | 11 |

## **Список таблиц**

# 1 Цель работы

Изучить задачу дискретного логарифмирования в конечных полях и её применение в криптографии с открытым ключом. Реализовать программно  $\rho$ -метод Полларда для нахождения показателя  $x$  в сравнении  $a^x \equiv b \pmod{p}$ , понять принципы поиска коллизий и решения линейных сравнений в циклических группах.

## 2 Модуль реализации алгоритма

### 2.1 Модуль реализации алгоритма

$\rho$ -метод Полларда для дискретного логарифмирования основан на поиске совпадений (коллизий) в последовательности элементов группы. Для этого используется специальное случайное отображение  $f$ , которое должно обладать свойством «вычислимости логарифма». Это означает, что если текущий элемент последовательности имеет вид  $c = a^u b^v \pmod{p}$ , то для следующего элемента  $f(c)$  также должны быть известны коэффициенты в его представлении через  $a$  и  $b$ . При обнаружении коллизии  $c_i = c_j$  задача сводится к решению линейного сравнения относительно неизвестного  $x$ .

### 2.2 Шаги реализации:

- Инициализация:
  - Выбрать произвольные целые числа  $u$  и  $v$ .
  - Вычислить начальное значение  $c = a^u b^v \pmod{p}$ .
  - Установить два указателя: медленный ( $c$ ) и быстрый ( $d$ ), где изначально  $d = c$ .
- Итерационный цикл:
  - Обновлять значения с помощью ветвящегося отображения  $f(c)$ .

- На каждом шаге медленный указатель делает один шаг ( $c = f(c)$ ), а быстрый — два ( $d = f(f(d))$ ).
  - Одновременно вычислять логарифмы для  $c$  и  $d$  как линейные функции от  $x$ :  $u + v \cdot x \pmod{r}$ , где  $r$  — порядок числа  $a$
  - Продолжать до тех пор, пока не будет получено равенство  $c \equiv d \pmod{p}$ .
- Решение сравнения:
    - Приравнять логарифмы, полученные при коллизии:  $u_c + v_c \cdot x \equiv u_d + v_d \cdot x \pmod{r}$ .
    - Решить полученное линейное сравнение первой степени относительно  $x$ .
    - Результат: найденное значение  $x$  или сообщение «Решений нет».

## 2.2.1 Модуль реализации алгоритма компилятора

```
import math

def extended_gcd(a, b):
    """
    扩展欧几里得算法，用于求解线性同余方程。
    Расширенный алгоритм Евклида для рекурсии
    """
    if a == 0:
        return b, 0, 1
    d, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return d, x, y

def solve_linear_congruence(A, B, M):
    """
    求解  $Ax \equiv B \pmod{M}$  类型的方程。
    Решение сравнения вида  $Ax \equiv B \pmod{M}$ .
    """
    d, x0, y0 = extended_gcd(A, M)
    if B % d != 0:
        return None # Решений нет (无解)
    x0 = (x0 * (B // d)) % (M // d)
    return x0
```

Рис. 2.1: Модуль реализации алгоритма компилятора



## 2.2.2 Модуль реализации алгоритма компилятора

```
def pollard_rho_discrete_log(p, a, r, b):  
    """  
    离散对数波拉德 rho 方法。  
    p-Метод Полларда для задачи дискретного л  
    """  
    # 按照实验定义的分段映射函数 f(c)  
    # Определение ветвящейся функции f(c) со г л а  
    def f(c, u, v):  
        # 实验报告示例中使用 p/2 作为分界点  
        # В примере из отчета p/2 используется  
        if c < p // 2:  
            # f(c) = ac, log_a(f(c)) = log_a(c) + 1  
            return (a * c) % p, (u + 1) % r, v % r  
        else:  
            # f(c) = bc, log_a(f(c)) = log_a(c) + x  
            return (b * c) % p, u % r, (v + 1) % r  
  
    # 1. 选择随机初值 u, v 并计算 c  
    # 1. Выбрать произвольные целые числа u, v и  
    u_c, v_c = 2, 2 # 采用实验报告中的初值  
    c = (pow(a, u_c, p) * pow(b, v_c, p)) % p  
  
    u_d, v_d = u_c, v_c  
    d = c  
  
    print(f"{'Step':<6} {'c':<10} {'log_a(c)':<15} {'d':<10} {'log_a(d)':<15}")  
    print("-" * 60)
```

Рис. 2.2: Модуль реализации алгоритма компилятора

### 2.2.3 Модуль реализации алгоритма компилятора

```
# 2. 执行步进直到  $c \equiv d \pmod{p}$ 
# 2. Выполнять шаги до получения равенства
# 使用 Floyd 判圈算法 (龟兔赛跑)
for i in range(1, 1000): # 设置最大步数防止死循环
    # c 每次走一步
    c, u_c, v_c = f(c, u_c, v_c)
    # d 每次走两步
    d, u_d, v_d = f(d, u_d, v_d)
    d, u_d, v_d = f(d, u_d, v_d)

    print(f"{i:<6} {c:<10} {u_c}+{v_c}x{' '}<6} {d:<10} {u_d}+{v_d}x")

    if c == d:
        # 3. 建立线性方程:  $u_c + v_c x \equiv u_d + v_d x \pmod{r}$ 
        # 3. Приравняв логарифмы, вычислить
        A = (v_d - v_c) % r
        B = (u_c - u_d) % r

        x = solve_linear_congruence(A, B, r)
        return x

return "Решений нет"
```

Рис. 2.3: Модуль реализации алгоритма компилятора

### 2.2.4 результат

|                                                                                                                                                                                                                                                                                                                                                                                                    |    |          |    |          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----------|----|----------|
| <pre># 样例 (Пример): # 使用实验报告中的数据: 10^x ≡ 64 (mod 107), r = 53 # 使用实验报告中的数据: 10^x ≡ 64 (mod 107), r = 53 p_val = 107 a_val = 10 r_val = 53 b_val = 64  result = pollard_rho_discrete_log(p_val, a_val, r_val, b_val)  print("-" * 60) print(f"结论 (Результат): x = {result}") if result is not None:     print(f"验证 (Проверка): {a_val}^{result} ≡ {pow(a_val, result, p_val)} (mod {p_val})")</pre> |    |          |    |          |
| Step                                                                                                                                                                                                                                                                                                                                                                                               | c  | log_a(c) | d  | log_a(d) |
| 1                                                                                                                                                                                                                                                                                                                                                                                                  | 40 | 3+2x     | 79 | 4+2x     |
| 2                                                                                                                                                                                                                                                                                                                                                                                                  | 79 | 4+2x     | 56 | 5+3x     |
| 3                                                                                                                                                                                                                                                                                                                                                                                                  | 27 | 4+3x     | 75 | 5+5x     |
| 4                                                                                                                                                                                                                                                                                                                                                                                                  | 56 | 5+3x     | 3  | 5+7x     |
| 5                                                                                                                                                                                                                                                                                                                                                                                                  | 53 | 5+4x     | 86 | 7+7x     |
| 6                                                                                                                                                                                                                                                                                                                                                                                                  | 75 | 5+5x     | 42 | 8+8x     |
| 7                                                                                                                                                                                                                                                                                                                                                                                                  | 92 | 5+6x     | 23 | 9+9x     |
| 8                                                                                                                                                                                                                                                                                                                                                                                                  | 3  | 5+7x     | 53 | 11+9x    |
| 9                                                                                                                                                                                                                                                                                                                                                                                                  | 30 | 6+7x     | 92 | 11+11x   |
| 10                                                                                                                                                                                                                                                                                                                                                                                                 | 86 | 7+7x     | 30 | 12+12x   |
| 11                                                                                                                                                                                                                                                                                                                                                                                                 | 47 | 7+8x     | 47 | 13+13x   |
| 结论 (Результат): x = 20                                                                                                                                                                                                                                                                                                                                                                             |    |          |    |          |
| 验证 (Проверка): 10^20 ≡ 64 (mod 107)                                                                                                                                                                                                                                                                                                                                                                |    |          |    |          |

Рис. 2.4: Модуль реализации алгоритма результат

### 3 Вывод

В ходе выполнения работы был успешно реализован  $\rho$ -метод Полларда для задачи дискретного логарифмирования. На примере сравнения  $10^x \equiv 64 \pmod{107}$  было продемонстрировано нахождение коллизии на 11-м шаге вычислений. Путем решения линейного сравнения  $7 + 8x \equiv 13 + 13x \pmod{53}$  было получено значение  $x = 20 \pmod{53}$ . Проверка подтвердила правильность результата:  $10^{20} \equiv 64 \pmod{107}$ . Алгоритм показал свою эффективность, сводя сложную задачу логарифмирования к поиску совпадений в последовательности и решению простых сравнений.